TESTING NEURAL NETWORKS ON MOBILE DEVICES

Hung Manh Nguyen

Bachelor's thesis
Faculty of Information Technology
Czech Technical University in Prague
Department of Applied Mathematics
Study program: Informatics
Specialisation: Artificial intelligence
Supervisor: Ing. Jan Čech, Ph.D.
May 11, 2026

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Nguyen** Jméno: **Hung Manh** Osobní číslo: **515728**

Fakulta/ústav: **Fakulta informačních technologií**

Zadávající katedra/ústav: **Katedra aplikované matematiky**

Studijní program: **Informatika**

Specializace: **Umělá inteligence**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Testování neuronových sítí na mobilních zařízeních

Název bakalářské práce anglicky:

Testing Neural Networks on Mobile Devices

Jméno a pracoviště vedoucí(ho) bakalářské práce:

Ing. Jan Čech, Ph.D. skupina vizuálního rozpoznávání FEL

Jméno a pracoviště druhé(ho) vedoucí(ho) nebo konzultanta(ky) bakalářské práce:

Datum zadání bakalářské práce: **13.02.2026** Termín odevzdání bakalářské práce: ____________

podpis vedoucí(ho) ústavu/katedry

podpis děkana(ky)

III. PŘEVZETÍ ZADÁNÍ

Student bere na vědomí, že je povinen vypracovat bakalářskou práci samostatně, bez cizí pomoci, s výjimkou poskytnutých konzultací. Seznam použité literatury, jiných pramenů a jmen konzultantů je třeba uvést v bakalářské práci.

Datum převzetí zadání

Nguyen Hung Manh

Podpis studenta

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

I. OSOBNÍ A STUDIJNÍ ÚDAJE

Příjmení: **Nguyen** Jméno: **Hung Manh** Osobní číslo: **515728**

Fakulta/ústav: **Fakulta informačních technologií**

Zadávající katedra/ústav: **Katedra aplikované matematiky**

Studijní program: **Informatika**

Specializace: **Umělá inteligence**

II. ÚDAJE K BAKALÁŘSKÉ PRÁCI

Název bakalářské práce:

Testování neuronových sítí na mobilních zařízeních

Název bakalářské práce anglicky:

Testing Neural Networks on Mobile Devices

Pokyny pro vypracování:

The goal of this thesis is to experimentally evaluate the feasibility of running modern neural networks for face processing on mobile devices (Android, iOS). The student will compare different architectures, frameworks, and device types in terms of speed, energy consumption, and accuracy. The work will focus primarily on real-time face detection and facial landmark estimation from video.

Tasks

1. Familiarize yourself with the options for running neural networks on mobile platforms.
2. Select suitable models for facial landmark detection and collect several test videos.
3. Implement a testing scenario on both Android and iOS.
4. Evaluate latency (ms/frame), throughput (fps), and optionally energy consumption. Compare results across several different devices.
5. Optionally, explore various optimization methods (e.g., quantization) and evaluate their impact on speed and accuracy.

References

[1] Li, Zhuojin, Marco Paolieri, and Leana Golubchik. "A benchmark for ML inference latency on mobile devices." Proceedings of the 7th International Workshop on Edge Systems, Analytics and Networking. 2024.

[2] Lee, Juhyun, et al. "On-device neural net inference with mobile GPUs." arXiv preprint arXiv:1907.01989 (2019).

[3] Wang, Siqi, Anuj Pathania, and Tulika Mitra. "Neural network inference on mobile SoCs." IEEE Design & Test 37.5 (2020): 50-57.

[4] Suchanek, Matej. "Efficient Implementation of Neural Networks for Real-Time Applications." Bachelor's thesis. Czech Technical University in Prague, 2020.

Seznam doporučené literatury:

Citation of this thesis: Nguyen Hung Manh. *Testing Neural Networks on Mobile Devices.* Bachelor's thesis. Czech Technical University in Prague, Faculty of Information Technology, 2026.

I would like to thank my supervisor Ing. Jan Čech, Ph.D. for his guidance, feedback, and support throughout this thesis.
I also thank my family and friends for their encouragement.

Declaration

I hereby declare that the presented thesis is my own work and that I have cited all sources of information in accordance with the Guideline for adhering to ethical principles when elaborating an academic final thesis.

I acknowledge that my thesis is subject to the rights and obligations stipulated by the Act No. 121/2000 Coll., the Copyright Act, as amended, in particular that the Czech Technical University in Prague has the right to conclude a license agreement on the utilization of this thesis as a school work under the provisions of Article 60 (1) of the Act.

I declare that I have used AI tools during the preparation and writing of my thesis. I have verified the generated content. I confirm that I am aware that I am fully responsible for the content of the thesis.

In Praze on May 11, 2026

Abstract

This thesis studies deployment and evaluation of neural networks for facial landmark detection on mobile devices. It explains mobile hardware constraints, efficient vision-model design, post-training quantization, and the ExecuTorch runtime. The practical part designs comparable iOS and Android benchmark applications that primarily measure landmark-model forward-pass latency and also report landmark accuracy on AFW-based inputs. The experiments compare XNNPACK, Core ML, and portable ExecuTorch backends on a PC, iPhone 14, and Xiaomi Redmi Note 8 Pro. The results show that backend choice and quantization have a strong effect on landmark latency, but the benefit of INT8 depends on the architecture and does not always transfer directly to the full detection pipeline.

Keywords mobile inference, face detection, facial landmarks, benchmarking, quantization, ExecuTorch, Core ML, XNNPACK

Abstrakt

Tato práce se zabývá nasazením a vyhodnocením neuronových sítí pro detekci obličejových landmarků na mobilních zařízeních. Popisuje omezení mobilního hardwaru, principy efektivních architektur, post-tréninkové kvantizace a runtime ExecuTorch. Praktická část navrhuje srovnatelný benchmark pro iOS a Android, který měří především latenci dopředného průchodu landmark modelu a současně sleduje přesnost landmarků nad daty AFW. Experimenty porovnávají backendy XNNPACK, Core ML a portable ExecuTorch na počítači, iPhonu 14 a zařízení Xiaomi Redmi Note 8 Pro. Výsledky ukazují, že volba backendu a kvantizace výrazně ovlivňuje latenci landmark modelu, ale přínos INT8 závisí na konkrétní architektuře a nemusí se přímo promítnout do celé detekční pipeline.

Klíčová slova mobilní inference, detekce obličeje, obličejové body, benchmark, kvantizace, ExecuTorch, Core ML, XNNPACK

Contents

List of Figures

List of Tables

List of abbreviations

AFW	Annotated Faces in the Wild (dataset)
AI	Artificial intelligence
ANE	Apple Neural Engine
AOT	Ahead-of-time
API	Application programming interface
AUC	Area Under the Curve
BS	Batch size
CNN	Convolutional Neural Network
CPU	Central Processing Unit
Core ML	Apple on-device machine learning framework
CSV	Comma-separated values
E2E	End-to-end
FLOPs	Floating-point operations
FP32	32-bit floating point
FPS	Frames per second
GPU	Graphics Processing Unit
INT8	8-bit integer quantization
IoU	Intersection over Union
JNI	Java Native Interface
LM	Landmark
NME	Normalized Mean Error
NPU	Neural Processing Unit
PC	Personal computer
PFLD	Practical Facial Landmark Detector
PIPNet	Pixel-in-Pixel Network
PTQ	Post-training quantization
PT2E	PyTorch 2 Export quantization flow
QAT	Quantization-aware training
ROI	Region of interest
SoC	System-on-Chip
XNNPACK	Optimized CPU execution path used by ExecuTorch

Chapter 1
Introduction

This thesis studies deployment and evaluation of neural networks for facial landmark detection on mobile devices. It combines background on mobile inference, efficient models, quantization, and ExecuTorch with a reproducible benchmark for iOS and Android. The benchmark focuses on landmark-model forward-pass latency and accuracy for multiple backend and quantization configurations.

1.1 Motivation

On-device inference has become a key requirement for many applications, because it improves privacy, reduces round-trip latency, and allows offloading compute resource to edge devices. At the same time, mobile devices are constrained by strict limits on compute throughput, memory bandwidth, power, and thermal budget [1, 2]. In practice, this means that achieving good model accuracy is not sufficient; the full end-to-end system must also be efficient.

Face processing usually starts with detection. Before an application can analyze a face, it first has to decide whether a face is present and where it is in the image. This detection step turns a full camera frame into a smaller region of interest, which reduces later computation and gives the rest of the pipeline a well-defined input crop. For many applications, however, a bounding box is only a coarse answer: it says where the face is, but not how the face is oriented, how its parts are arranged, or how its expression changes.

Facial landmarks add this finer information. Instead of representing a face only by a rectangle, they mark points such as eye corners, nose position, mouth corners, and jaw shape. These points make it possible to align faces before face recognition or verification [3], estimate head pose and localize faces in difficult real-world images [4], analyze facial expression, drive avatar or face-animation systems, and place augmented-reality effects such as glasses, masks, or makeup on the correct parts of the face [5]. In medical settings, landmark-based facial analysis can also help quantify facial asymmetry, support facial-palsy assessment, and track recovery or rehabilitation progress from facial movement changes [6].

Recent community benchmarks confirm that measured latency strongly depends on hardware configuration, runtime backend, and numerical precision [7, 2].

Figure 1.1 Face-processing pipeline on a real Annotated Faces in the Wild (AFW) sample image [4]: the first panel shows the input crop, the second panel draws the face region derived from the AFW landmark annotation, and the third panel overlays the annotated landmark points from the corresponding `.pts` file.

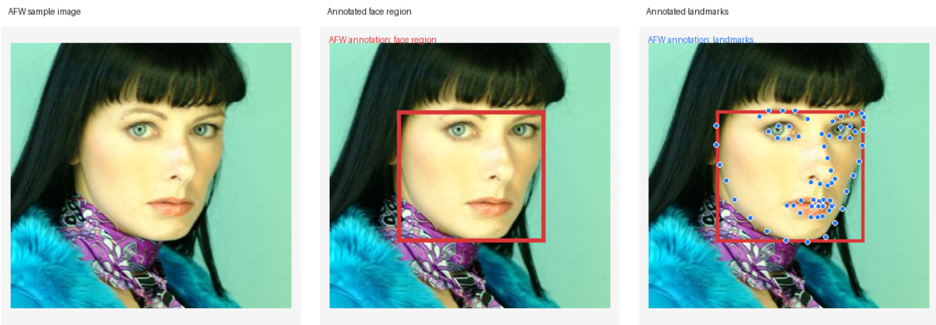


Figure 1.1 shows the practical meaning of the two-stage task before any performance numbers are discussed. The first panel is the image as it enters the application. The second panel shows the face region: the rectangle is derived from the AFW landmark annotation for the sample image [4]. A deployed system would obtain this region from a detector before cropping and normalizing the face. The third panel shows the annotated landmark points from the corresponding `.pts` file. These semantically meaningful points, such as eyes, nose, mouth, and face contour positions, are what make downstream tasks such as face alignment, expression analysis, augmented-reality overlays, and pose estimation possible.

1.2 Problem statement

The core of this thesis is to evaluate neural networks for detecting facial landmarks on mobile devices, with a focus on configurations implemented with ExecuTorch. The benchmark must support equivalent iOS and Android applications, consistent measurement conditions, and validation of quality metrics.

The benchmark is a two-stage pipeline:

1. face detection in an input image,
2. facial landmark estimation on detected face crops.

1.3 Research objectives

Objectives of the thesis are:

- summarize practical options for running neural networks on Android and iOS;
- select suitable face-processing models (detector + landmark regressor) for mobile real-time use;
- implement equivalent benchmark scenarios on both platforms;
- evaluate latency (ms/frame), throughput (frames per second, FPS), and landmark accuracy;
- analyze post-training quantization (PTQ) with 8-bit integer (INT8) optimization impact on speed and quality.

1.4 Alignment with thesis assignment tasks

The assignment tasks are covered as follows:

1. **Mobile neural network (NN) deployment options:** Chapter 2 and Chapter 3 review mobile hardware/runtime constraints and ExecuTorch backend options.
2. **Model selection + test inputs:** Chapter 2 (model section) explains selected detector/landmark architectures; Chapter 4 defines the reproducible AFW (Annotated Faces in the Wild)-based input protocol [4] used for quantitative comparison.
3. **Android/iOS testing scenario:** Chapter 4 documents feature parity and unified measurement schema for both mobile apps.
4. **Latency/FPS and optional energy:** Chapter 4 defines metrics and logging; Chapter 5 reports results measured on various devices.
5. **Optimization impact (quantization):** Chapter 3 explains PTQ export workflow and Chapter 5 compares 32-bit floating point (FP32) vs INT8 behavior.

Chapter 2

Theoretical background

2.1 On-device inference in mobile environments

Mobile artificial intelligence (AI) systems are typically deployed on heterogeneous Systems-on-Chip (SoCs), where central processing units (CPUs), graphics processing units (GPUs), and dedicated neural processing units (NPUs) coexist [8, 1]. Compared to server inference, mobile workloads operate under tighter thermal and power limits and often share resources with other tasks.

Latency depends on several things:

- model architecture (operator types, tensor shapes, parameter count),
- runtime backend (kernel library, operator fusion, threading policy),
- hardware mapping (CPU, GPU, and NPU dispatch),

That means the same neural network can show very different performance across devices and software stacks [2, 7].

2.2 Hardware and system constraints

2.2.1 CPU, GPU, and NPU roles

CPUs provide broad operator coverage and predictable behavior, but may not offer the highest throughput for dense tensor computations. Mobile GPUs can improve throughput for selected workloads, while NPUs may deliver superior energy-efficiency when supported operators map well to accelerator kernels [8, 1]. In real applications, unsupported subgraphs frequently fall back to CPU, and thus synchronization and memory transfer overhead become issues.

The difference between these three hardware types is important for interpreting mobile benchmark results. A CPU is the most general processor in the phone. It is good at control flow, irregular work, small operations, and operators that are not supported by accelerators. This makes it reliable, but not always the fastest choice for large neural-network tensor operations. In this thesis, XNNPACK (optimized CPU backend) represents the optimized CPU path, so XNNPACK results should be understood as optimized CPU execution rather than GPU or NPU execution [9, 10].

A GPU is designed for many similar operations in parallel. Neural-network layers such as convolutions and matrix multiplications can often be expressed as large parallel workloads, so a GPU can be faster than a CPU for supported operators. The drawback is that GPU execution has overhead: data may need to be prepared for the GPU, command queues must be scheduled, and unsupported operators may have to move back to the CPU. For small models or small batch sizes, this overhead can reduce the practical speedup [8, 2].

An NPU is a dedicated accelerator for neural-network operations. It is usually designed to run supported layers with high energy efficiency. The limitation is operator coverage: if the model contains an unsupported layer, unsupported shape, or unsupported data type, the runtime may fall back to CPU execution for that part. This fallback can introduce extra copies and synchronization, so an accelerator is useful only when a large enough part of the graph maps cleanly to it [1, 7].

In this thesis, the measured backends are Core ML (Apple machine learning framework), XNNPACK, and portable ExecuTorch execution. Core ML can choose Apple CPU, GPU, or Apple Neural Engine (ANE) depending on the model and device [11, 12]. XNNPACK is the optimized CPU path [9]. Portable execution is the generic ExecuTorch path used mainly as a correctness baseline when no backend-specific acceleration is used [13, 14].

2.2.2 Power and thermal behavior

Peak performance is often not sustainable in mobile devices. Very long inference can trigger thermal throttling, reducing frequency and increasing latency [2].

2.3 Efficient neural networks for mobile vision

The shift from convolutional neural networks (CNNs) deployed on servers to mobile-friendly architectures was enabled by operator-level efficiency ideas such as depthwise separable convolution, inverted residual blocks, and channel shuffle [15, 16, 17]. SqueezeNet demonstrated that competitive accuracy can be achieved with significantly fewer parameters [18].

For processing faces on mobile devices, both detector and landmark models must be efficient.

2.4 Model compression and quantization

Compression and quantization reduce memory footprint and often improve runtime performance. The most common compression techniques include pruning and weight sharing [19]. In practical mobile deployment, integer quantization is especially important because INT8 kernels can be faster and more energy-efficient than FP32 on supported hardware [20].

2.4.1 Post-training quantization

Post-training quantization converts an already trained FP32 model to a lower-precision representation, usually without retraining. In this thesis, the main target is INT8 execution. A real value x is represented by an integer q using scale s and zero-point z:

$$q = \mathrm{clip}\left(\mathrm{round}\left(\frac{x}{s}\right) + z,\ q_{\min}, q_{\max}\right), \qquad \hat{x} = s\,(q - z). \tag{2.1}$$

Here, $q_{\min}$ and $q_{\max}$ are the lower and upper integer bounds of the target quantized type, round maps to the nearest integer, and clip clamps the rounded value to this representable range. For signed INT8, q is typically in $[-128, 127]$ (or $[-127, 127]$ in symmetric weight quantization). The reconstructed value $\hat{x}$ approximates x, so a small quantization error is introduced.

The scale s is the size of one integer step in the original real-number space. If s is small, the integer grid is fine and precise, but the representable range is narrow. If s is large, the range is wide, but nearby real values are rounded to the same integer. The zero-point z says which integer represents real value zero. In symmetric quantization, z is normally zero or very close to zero, which makes signed integer arithmetic simpler [20]. In asymmetric quantization, z can shift the integer range so that non-symmetric activation ranges can be represented more efficiently [21].

Calibration is the step that chooses useful ranges for tensors. During calibration, representative input samples are passed through the prepared model, and observer nodes record approximate minimum and maximum values of activations [22]. These observed ranges are then converted into scales and zero-points. If calibration data is too narrow, later real inputs may fall outside the observed range and get clipped by the clip operation in the equation above. If calibration data is too wide because of rare outliers, the scale becomes too large and normal values lose precision. This is why PTQ must be evaluated experimentally instead of being assumed to preserve accuracy.

For weights, per-tensor quantization uses one scale for the whole tensor. Per-channel quantization uses a separate scale for each output channel, so one channel with large values does not force all other channels to use a coarse scale [20]. For convolution and linear layers, this is often important because different output channels can have very different value ranges.

PTQ typically has three steps:

1. prepare the graph for quantization,
2. run calibration data through the model to estimate activation ranges,
3. convert operators to quantized kernels.

Compared to quantization-aware training (QAT), PTQ is much simpler to apply, but it is more sensitive to calibration quality and to model-specific outliers.

2.4.2 INT8 design choices that affect quality

Two design choices are central for this work:

- **Per-tensor vs per-channel weights:** per-channel assigns separate scales per output channel and usually preserves accuracy better, especially in convolution-heavy models.
- **Static vs dynamic activations:** static PTQ calibrates activation ranges offline; dynamic approaches estimate part of the range online. Static PTQ is often faster when backend support is good.

INT8 often improves speed for two reasons: lower memory traffic (INT8 tensors are 4× smaller than FP32 tensors) and faster vectorized kernels. However, real inference latency reduction depends on operator coverage. If part of the graph falls back to non-quantized execution, conversion overhead can reduce the expected speedup [21].

2.5 Face detection and landmark estimation

2.5.1 Task definitions

Face detection finds faces with bounding boxes. Landmark estimation predicts keypoint coordinates (e.g., eye corners, nose tip, mouth corners), usually on a cropped face image. These tasks are strongly coupled: detection quality and crop geometry directly influence landmark quality.

2.5.2 Representative models

RetinaFace is a strong one-stage detector with high robustness under challenging conditions [23, 24]. FaceBoxes is a lightweight detector explicitly designed for CPU real-time operation [25, 26].

For landmarks, PFLD (Practical Facial Landmark Detector) emphasizes compactness and speed [5, 27]. MobileFaceNet provides a lightweight backbone originally developed for mobile face recognition and often reused for landmark regression [3, 28]. PIPNet (Pixel-in-Pixel Network) combines heatmap-style reasoning with low-resolution efficient heads for practical landmark localization [29, 30].

In the benchmark matrix used in this thesis, the main detector+landmark combinations are based on RetinaFace-MNet0.25 or FaceBoxesV2 with MobileFaceNet, PFLD-MNv2-0.25, PFLD-MNv2-1.0, and PipNet-ResNet18.

Table 2.1 Qualitative properties of models used in the benchmark matrix.

Model	Stage	Relative cost	Main properties
RetinaFace-MNet0.25	Detector	Medium	Robust detection quality; higher detector cost than very light alternatives.
FaceBoxesV2	Detector	Low	Very fast detector; useful when throughput is prioritized.
MobileFaceNet	Landmark	Low	Lightweight baseline; stable and efficient for mobile CPUs.
PFLD-MNv2-0.25	Landmark	Low	Compact model with good speed and moderate landmark precision.
PFLD-MNv2-1.0	Landmark	Medium	Higher capacity than 0.25 variant; better potential quality at higher compute cost.
PipNet-ResNet18	Landmark	High	Stronger landmark localization capacity, but substantially heavier compute load.

2.5.3 Datasets and annotation uncertainty

In this thesis, Annotated Faces in the Wild (AFW) annotations are used for wild-condition landmark evaluation, originating from the face detection and pose estimation work of Zhu and Ramanan [4].

Chapter 3

ExecuTorch for mobile deployment

3.1 ExecuTorch motivation

ExecuTorch is a PyTorch edge deployment stack designed for efficient on-device inference across mobile and embedded devices [13, 14]. Its central design principle is *ahead-of-time* (AOT) preparation: expensive transformation and planning steps are performed before deployment, so runtime execution remains lightweight.

ExecuTorch was chosen instead of TensorFlow Lite (TFLite) or another mobile runtime mainly because this project starts from PyTorch models and needs the same deployment idea on Android and iOS. TFLite is a strong mobile and embedded runtime for models in the TensorFlow Lite format [31], and ONNX Runtime Mobile is a strong mobile runtime for models represented in the Open Neural Network Exchange (ONNX) format [32]. However, using either of them here would add an extra conversion boundary: the model would first have to leave the PyTorch export path and then be validated again in another framework-specific graph format. That extra step is risky for this benchmark because small differences in operator conversion, tensor layout, unsupported operations, or numerical precision could change both speed and landmark accuracy.

ExecuTorch keeps the pipeline closer to PyTorch: `torch.export` captures the model graph, ExecuTorch lowers that graph for edge execution, and the final `.pte` file is loaded by the Android and iOS benchmark applications [33, 34, 35]. It also matches the backend comparison needed in this thesis. The same deployment stack can produce an optimized CPU path through XNNPACK, an Apple path through Core ML, and a portable ExecuTorch baseline [9, 12, 13]. This makes the benchmark easier to interpret: differences in the results are more directly connected to backend choice, quantization, device hardware, and model architecture instead of being mixed with a completely different model-conversion toolchain.

3.2 Conceptual architecture

ExecuTorch organizes model deployment into three phases [13, 14]:

1. export a PyTorch model to graph form,
2. lower and optimize the graph for edge execution,
3. execute the serialized program with a small C++ runtime.

The first phase, export, captures the model as a graph. A graph is a list of tensor operations and their data dependencies. This graph form is important because it can be checked, transformed, quantized, partitioned, and serialized [33, 34].

In more detail, the graph is a data-flow description of the neural network. Each node represents something that must happen, such as reading an input tensor, reading a trained weight tensor, running a convolution, applying an activation function, reshaping a tensor, or returning the final output. The edges between nodes represent values flowing from one operation to another [33]. If node B uses the result of node A, then the graph records that dependency. This lets the compiler know that A must run before B, and it also lets the compiler see when the output of A is no longer needed.

The graph is an internal program representation. It contains placeholders for model inputs, constants or parameters for learned weights, operator calls, metadata about tensor shapes, and output nodes [33, 13]. A tensor is the multi-dimensional array used by neural networks. For example, an image batch may be represented as a tensor with dimensions for batch, height, width, and channels. Operator nodes consume tensors and produce new tensors. A convolution node consumes an input feature tensor and a weight tensor, then produces an output feature tensor. The next activation node consumes that output and produces another tensor.

This representation is useful because it makes implicit model behavior explicit. In normal PyTorch eager execution, Python code decides what operation to run next while the model is executing. In an exported ExecuTorch workflow, that decision has already been captured into the graph [13]. The phone therefore does not need to interpret the original Python model. It only follows the prepared graph program.

Technically, `torch.export` captures an Export Intermediate Representation, often called Export IR, that preserves the meaning of the original PyTorch program while removing the need to run the original Python control code on the phone [33, 13]. The exported graph records operators, tensor shapes or shape constraints, parameters, and the order in which values flow between operators [33]. This is why ExecuTorch can analyze the model before deployment instead of discovering the model structure during each mobile inference run.

Shape information is part of why the graph can be compiled. If an input always has shape $1 \times 3 \times 112 \times 112$, the compiler can plan memory for that exact shape. If the batch dimension is dynamic, the graph can store a constraint such as "batch is between 1 and 128" instead of a single fixed number [33]. ExecuTorch can then use these constraints during lowering, memory planning, and backend partitioning.

After export, compiler passes can rewrite the graph. A pass is a transformation that reads the graph and produces a modified graph or program. Typical passes can decompose high-level PyTorch operators into simpler Core ATen operators, insert or remove quantization-related operations, identify which nodes can be handled by XNNPACK or Core ML, and assign temporary tensors to planned memory locations [13, 14, 36, 37]. Because the model is represented as nodes and edges, these transformations can be done systematically rather than by manually editing model code.

The second phase, compile or lower, turns the exported graph into an ExecuTorch program. During this phase ExecuTorch can decompose operators into a smaller standardized operator set, lower supported graph regions to backends such as XNNPACK or Core ML, and plan temporary tensor memory [14, 36, 37].

ExecuTorch uses operator standardization to make this possible. Many PyTorch operators are decomposed into a smaller Core ATen operator set, where ATen is PyTorch's core tensor and operator library, and the portable operator library implements this standardized set when no delegate handles the operation [13, 14]. This matters because backend authors do not need to support the entire Python-level PyTorch API; they only need to support the graph operators that the exported model contains.

The third phase, execution, is intentionally small. The runtime reads the serialized program, prepares memory, calls either portable kernels or delegate backends, and returns output values. The runtime is written in C++ and is wrapped by platform application programming interfaces (APIs) for Swift and Kotlin/Java [13, 38, 39].

The runtime is also designed so that only the operators needed by the deployed program have to be linked into the final app binary [13]. In simple terms, the phone does not ship a full Python interpreter or a full training framework. It ships a small execution engine, the required kernels, and any registered delegate backends needed by the exported model.

3.3 Export and lowering workflow

The practical workflow consists of:

1. model export using `torch.export`,
2. backend specific lowering via `to_edge_transform_and_lower`,
3. creation of a serialized `.pte` artifact, where an artifact means a file produced by the build/export pipeline and used later by another part of the project.

In this thesis, a model artifact is usually a `.pte` file: it is produced on the development machine, copied into the Android or iOS application, and loaded by the mobile benchmark.

This process supports dynamic shape constraints and optional separation of program and tensor data into `.pte` and `.ptd` files [34, 35, 40, 33].

3.3.1 Intermediate representations used before runtime

ExecuTorch does not directly run the original Python model on the phone. The model passes through intermediate representations, meaning machine-readable forms of the model that are easier for compiler passes to analyze and modify [14]. The important idea is:

- **PyTorch model**: the normal Python model used during development;
- **Exported graph**: a graph produced by `torch.export`, with tensor operations and shape constraints [33];
- **Edge program**: an ExecuTorch-friendly program after edge transformations, backend lowering, and memory planning [34, 37];
- **Serialized artifact**: the final `.pte` (portable tensor executable) file copied to Android assets or the iOS bundle [35].

This is why the benchmark applications do not contain PyTorch model code. They only load already exported artifacts and feed tensors into them. In simple language, Python is used to prepare the model, and the phone only receives the finished program.

3.3.2 The `.pte` deployment artifact

The `.pte` file is the serialized ExecuTorch program used at runtime [35]. It contains the graph-level execution plan, operator references, delegate information, constants, and metadata needed by the ExecuTorch runtime. If large tensor data is separated from the program, a `.ptd` file can store that tensor data separately [40]. This thesis mainly treats `.pte` files as the deployable model artifacts: the Android and iOS applications load them, create input tensors, run forward execution, and read output tensors.

A `.pte` file is therefore not the same thing as a PyTorch checkpoint. A checkpoint stores trained parameters, and often assumes that Python model code will rebuild the network structure. A `.pte` file stores an already exported and lowered execution program. That program can include delegate segments, portable fallback segments, constants, and the method entry points that the runtime can call [35, 36].

3.3.3 Static shapes, dynamic shapes, and batching

Mobile runtimes are easier to optimize when tensor shapes are known in advance. A static-shape export fixes dimensions such as batch size and input height and width before deployment. A dynamic-shape export allows selected dimensions to vary inside declared bounds using `torch.export.Dim` constraints [33]. In this thesis, dynamic batch export means that the batch dimension can vary between a minimum and maximum value while image shape and channel count remain fixed. This allows one artifact to cover several batch sizes, but it also gives the compiler less exact shape information than a fully static export.

3.3.4 Memory planning

ExecuTorch performs memory planning during export and lowering so that the runtime can reuse buffers for temporary tensors [37]. This matters on mobile devices because memory allocation during repeated inference can add overhead and increase memory pressure. A simple way to think about it is that many intermediate tensors are not needed at the same time. If tensor A is no longer used before tensor B is created, both can reuse the same memory region. Planning this before runtime helps make execution more predictable.

3.4 Delegation and partitioning

ExecuTorch uses delegates to offload supported subgraphs to backend specific implementations [36]. A partitioner tags lowerable graph regions; unsupported regions remain in portable execution. This mixed execution model improves portability but can create overhead.

For this thesis, two backends are central:

- **XNNPACK**: optimized CPU backend with broad mobile support, including quantized paths [9, 41, 10];
- **Core ML delegate**: Apple backend path that can dispatch to CPU, GPU, or ANE depending on model and platform capabilities [12, 11].

A partitioner is the component that decides which graph regions a delegate can own [36]. If a sequence of operators is supported by XNNPACK, that region is marked for XNNPACK. If one operator in the middle is not supported, ExecuTorch may split the graph into multiple delegated regions with a portable fallback region between them [36]. This is important for the results chapter because the fastest model is not always the model with the fewest theoretical floating-point operations (FLOPs); backend coverage and fallback boundaries also matter.

Delegation is not just a runtime switch. It is decided during export and lowering. The partitioner examines the graph, checks backend support, and replaces supported graph regions with calls into a backend-specific lowered representation [36]. At runtime, ExecuTorch follows the already prepared execution plan. This is why two `.pte` files from the same neural network can behave differently: one may contain XNNPACK-delegated regions, one may contain Core ML-delegated regions, and one may contain only portable execution.

3.4.1 XNNPACK

XNNPACK is the CPU acceleration path used on both Android and iOS. It provides optimized neural network operators for mobile and desktop processors [10]. In ExecuTorch, the XNNPACK backend lowers supported graph regions into XNNPACK delegate calls [9]. This is useful for this thesis because it provides a common backend across Android and iOS, so the benchmark can compare the same exported family of artifacts on both platforms.

XNNPACK is a low-level library of optimized neural-network operator kernels, not a full training framework [10]. It targets CPU instruction sets on Arm and x86 systems, including Android, iOS, macOS, Linux, Windows, and simulators [10, 9]. The operators relevant to vision models include convolution, depthwise convolution, pooling, resize, fully connected layers, elementwise arithmetic, activation functions such as clamp/ReLU, sigmoid, tanh, and PReLU, and layout-related operations such as transpose [10]. This operator coverage is why XNNPACK is a reasonable backend for face detectors and landmark regressors, which are mostly made from convolution, activation, pooling, and linear layers.

The important technical point is that XNNPACK accelerates CPU execution by using specialized kernels and vectorized instructions. It does not move the model to the GPU or NPU. XNNPACK also commonly uses NHWC tensor layout, where the tensor dimensions are ordered as batch, height, width, and channels [10]. If an exported graph already matches what XNNPACK supports, the partitioner can delegate large regions. If the graph contains an unsupported operator or unsupported shape pattern, ExecuTorch must keep that part in portable execution, which can add boundary overhead.

For quantized models, the ExecuTorch XNNPACK backend supports 8-bit symmetric weights with 8-bit asymmetric activations through the PT2E flow [41]. The official XNNPACK quantization flow uses `XNNPACKQuantizer`, `prepare_pt2e`, calibration with representative inputs, `convert_pt2e`, and then lowering with `XnnpackPartitioner` [41]. This matches the export script used in this thesis.

3.4.2 Core ML

Core ML is the Apple-specific delegate path [11]. In ExecuTorch, the Core ML backend lowers supported model regions to Apple's machine learning stack [12]. Depending on the model and device, Core ML may execute work on the CPU, GPU, or ANE [11]. This means that Core ML numbers in the results chapter should not be interpreted as CPU-only numbers.

In the ExecuTorch export flow, Core ML is selected by using the Core ML partitioner during the edge lowering step [12]. The partitioner marks graph regions that can be compiled for Core ML, and the resulting `.pte` file contains the information needed to call the registered Core ML delegate at runtime [12, 36]. No separate model conversion is performed inside the benchmark app; the app loads the already prepared ExecuTorch program.

Core ML supports dynamic dispatch to Apple CPU, GPU, and ANE and supports FP32 and 16-bit floating point (FP16) computation through the ExecuTorch backend [12]. This is why Core ML can be much faster than portable execution on iPhone. The measured time includes the full benchmark pipeline, but the actual landmark model forward pass may run on Apple acceleration hardware rather than only on the CPU. Core ML delegated models also depend on platform support: the ExecuTorch Core ML backend documents iOS 16 or newer as a target requirement for running delegated models [12].

Core ML and XNNPACK therefore answer different questions in the thesis. XNNPACK answers: "How fast is the optimized CPU backend that is available on both Android and iOS?" Core ML answers: "How fast can the Apple-specific delegate run when it is allowed to use Apple's ML stack?" Portable execution answers: "What happens without backend-specific acceleration?"

3.5 Quantization in ExecuTorch

ExecuTorch implements quantization through backend-specific configurations. These commonly use PT2E (PyTorch 2 Export) with torchao-based preparation and conversion [42, 22]. In this project, the XNNPACK INT8 export script produces the quantized landmark models using symmetric PTQ.

3.6 Runtime integration in the benchmark applications

On iOS, the thesis app uses the Swift ExecuTorch API (application programming interface). The app creates a module from a model path, creates a tensor with the expected shape, calls the module forward method, and reads the returned tensor values. This matches the official iOS integration pattern for loading and executing an ExecuTorch model [39].

On Android, the thesis app uses the Java/Kotlin ExecuTorch API (application programming interface). The app loads a module from an asset path, wraps input data in a tensor, passes the tensor through `EValue` (ExecuTorch value wrapper) objects, and calls `forward`. The Java API is a wrapper over native ExecuTorch code and uses JNI (Java Native Interface) to cross between Java/Kotlin and C++ runtime code [38].

The important practical consequence is that neither mobile application trains or exports a model. Training and export happen before deployment. The mobile applications only run inference, meaning they load the `.pte` (serialized ExecuTorch program), feed input tensors, measure time, and store outputs for later analysis.

Chapter 4

Benchmark methodology

4.1 Methodological goals

The benchmark design in this thesis follows two goals:

1. measure latency, throughput, and accuracy consistently on iOS and Android,
2. preserve raw artifacts needed for analysis of the results.

4.2 Evaluated pipeline

Each benchmark run executes an end-to-end pipeline:

1. detector preprocessing,
2. detector inference,
3. detector postprocessing (box decoding and filtering),
4. face crop generation,
5. landmark model preprocessing,
6. landmark inference,
7. landmark postprocessing.

The detector part is always executed with detector batch size one in the reported pipeline runs. This means one image is normalized, passed through the detector, and decoded at a time. The landmark part has a separate *resolved landmark batch size*; this is the BS (batch size) value shown in the result tables and charts.

For a measurement iteration, the app first runs detection for the input image(s) and stores all detected boxes. It then creates face crops from those boxes. Finally, the generated crops are appended to a landmark queue and processed in chunks of the configured landmark batch size. This is why the thesis reports detector, crop, landmark-inference, and end-to-end timings separately: they correspond to distinct parts of the measured pipeline.

The reported thesis comparisons use 10 warm-up iterations before measurement. Warm-up iterations execute the same pipeline but are excluded from the aggregated timing statistics. Their purpose is to make the measured iterations represent steady-state inference rather than one-time setup costs. During the first few passes, the runtime may load model weights into memory, initialize backend delegates, compile or specialize kernels, allocate reusable buffers, fill CPU caches, and trigger operating-system scheduling or power-state changes. If these first passes were included, the reported latency would mix normal inference time with initialization overhead and would exaggerate the cost of configurations that perform more setup work. After warm-up, the same model, backend, batch size, and pipeline are measured under conditions closer to how the application behaves after it has already started processing frames.

4.3 Model families and configuration space

The methodology supports multiple detector/landmark pairs and backend configurations. Typical factors include:

- detector choice (e.g., RetinaFace vs FaceBoxes),
- landmark architecture (e.g., PFLD, MobileFaceNet, PIPNet),
- precision variant (FP32 and INT8),
- backend delegate (XNNPACK, Core ML, and portable),
- resolved batch size.

A complete run record stores these factors explicitly, so later analysis does not depend on file naming conventions alone.

4.4 iOS and Android implementation parity

Both iOS and Android benchmark applications are designed to be feature-equivalent at the pipeline level. Equivalent behavior includes:

- identical preprocessing parameters,
- identical postprocessing logic and thresholds,
- consistent metric collection and export schema,
- consistent warm-up and measured iterations.

4.5 Dataset and inputs

Annotated Faces in the Wild (AFW)-based landmark annotations are used for accuracy evaluation [4].

4.6 Metrics

4.6.1 Landmark inference latency

Although each run executes the full detector+landmark pipeline, the thesis comparison charts are organized by the *landmark* configuration (model, backend, quantization, and resolved batch size). The primary timing metric is therefore landmark *inference* latency only:

$$\ell_i^{\mathrm{lm}} = t_i^{\mathrm{end}} - t_i^{\mathrm{start}}. \tag{4.1}$$

Here, i indexes a measured landmark-model forward pass, t_i^{start} and t_i^{end} are its start and end timestamps, and ℓ_i^{lm} is the resulting latency in milliseconds.

For each benchmark run r, the exported latency samples form the set $L_r = \{\ell_i^{\mathrm{lm}} \mid i \in I_r\}$, where I_r is the set of measured iterations in that run. The run is summarized by the mean latency, the median, and the 95th percentile $p_{95}(L_r)$. For charts, repeated runs with the same landmark model, backend, quantization, device, and resolved batch size form one configuration group g, with run set R_g. For any selected run-level statistic y_r, such as the run median, the plotted group value is

$$\tilde{y}_g = \mathrm{median}\{y_r \mid r \in R_g\}. \tag{4.2}$$

4.6.2 Throughput and batch scaling

The benchmark applications export `fps`. It is run-level benchmark throughput, not landmark-forward-pass-only FPS. The forward-pass behavior of the landmark network is reported separately by the landmark-inference latency metric defined above. This distinction matters because an optimized landmark model can have very low forward latency while the measured run FPS is still limited by detector execution, preprocessing, postprocessing, queue flushing, or other surrounding pipeline work.

Run throughput follows the exported timing summary:

$$\mathrm{FPS}_{\mathrm{run}} = \frac{N_{\mathrm{images}}}{T_{\mathrm{run}}}. \tag{4.3}$$

Here, FPS means frames per second, N_{images} is the number of processed images, and T_{run} is the measured benchmark-run wall-clock time.

Batch size one (BS=1) charts in the results chapter focus on median landmark-inference latency by landmark configuration. Batch-scaling charts plot median landmark-inference latency as a function of resolved landmark batch size ($b \in \{1, 2, 4, 8\}$ in the curated thesis set), because this directly shows how the landmark model forward pass responds to batching.

4.6.3 Landmark accuracy

Accuracy is recomputed offline from exported per-image predicted landmarks and AFW ground truth annotations [4]. For N evaluated images and K landmarks, inter-ocular normalized mean error (NME) is

$$\mathrm{NME}_{\mathrm{iod}} = \frac{1}{N}\sum_{n=1}^{N}\frac{1}{K}\sum_{k=1}^{K}\frac{\|\hat{\mathbf{p}}_{n,k} - \mathbf{p}_{n,k}\|_2}{d_n^{\mathrm{iod}}}, \tag{4.4}$$

where $\hat{\mathbf{p}}_{n,k}$ is the predicted two-dimensional position of landmark k in image n, $\mathbf{p}_{n,k}$ is the corresponding AFW ground-truth landmark position in the same image coordinate system, and d_n^{iod} is the inter-ocular distance for image n.

4.6.4 Cumulative error histograms

The result chapter also uses cumulative error histograms, sometimes called cumulative distribution curves. They are built from per-image errors rather than from a single average. For each evaluated image, the offline script computes one error value, for example NME or RMSE. The script then sorts these values from smallest to largest and draws a curve answering the question: "if this error threshold is allowed, what fraction of images is already good enough?"

For a set of per-image errors $E = \{e_n \mid n = 1, \ldots, N\}$, the cumulative curve value at threshold τ is

$$C(\tau) = \frac{1}{N}\sum_{n=1}^{N}\mathbf{1}[e_n \leq \tau], \tag{4.5}$$

where $\mathbf{1}[\cdot]$ is one when the condition is true and zero otherwise. In these plots, RMSE means root-mean-square landmark error.

The horizontal axis is the error threshold. The vertical axis is the fraction of images whose error is less than or equal to that threshold. To read such a chart, choose a value on the horizontal axis, move upward until you reach a curve, and then read the vertical value. If the point is at 0.80, then 80% of evaluated images have error no larger than the chosen threshold. Reading in the other direction is also useful: start at 0.50 on the vertical axis to find the median error, or at 0.95 to see how large the error must be before 95% of images are covered.

This representation is useful because average error can hide failures. Two models may have the same mean NME, but one can be reliable on almost every image while the other is very accurate on easy images and very poor on difficult images. In a cumulative histogram, the better curve is generally higher and more to the left: it means more images reach low error thresholds. A steep curve means consistent behavior. A curve that rises slowly or has a long right tail means that some images are outliers where the landmark prediction failed or became noticeably worse.

4.6.5 Optional energy metrics

If platform tooling allows reliable collection, energy-related signals can be logged as supplementary indicators. In this thesis, landmark-inference latency, throughput, and landmark accuracy remain primary. Energy metrics are very hard to obtain. They are not included in the main results chapter, but they are collected when possible and stored in the raw artifacts for future analysis.

4.7 Measurement protocol

To reduce noise and measurement bias, each benchmark condition follows a fixed protocol:

- dedicated warm-up iterations before timed measurement,
- fixed number of measured iterations,
- stable device state (charging policy, background app control, thermal preconditioning),
- separate logging for initialization, inference, and optional postprocessing stages.

4.8 Run artifacts and offline analysis

Every run exports a machine-readable summary plus raw samples. Offline analysis scripts recompute aggregate statistics and recompute accuracy from image predictions instead of trusting prefilled summary fields. This allows for more flexible and transparent analysis, as well as the possibility to add new metrics without rerunning benchmarks.

4.9 Threats to validity

Main threats and mitigations include:

- **Thermal drift**: controlled warm-up and repeated runs;
- **Backend asymmetry**: shared-configuration comparisons for cross-device claims;

- **Unsupported operators**: explicit reporting of delegate coverage and fallback paths;

Chapter 5
Experimental results

5.1 Result scope and source artifacts

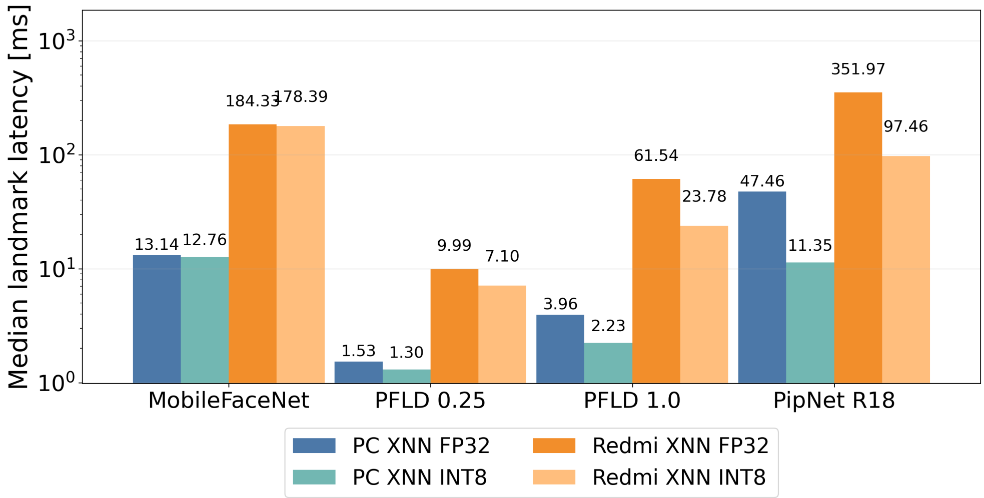
The main simplified dashboard is configured around one focus mobile device (Xiaomi Redmi Note 8 Pro) with personal computer (PC) reference rows. The PC is a MacBook Pro 14 with the M1 Pro chip and 14 core GPU and 16 GB of RAM. The Redmi phone is a mid-range Android device with an Arm Cortex-A76-based CPU and Mali-G76 GPU. The PC reference is included to show the gap between desktop and mobile execution, while Redmi is included to show the practical performance of the models on a real mobile device. The discussion focuses on landmark-model forward-pass latency; run FPS and end-to-end latency are secondary because they also include detector and application overhead.

5.2 Batch size 1 landmark forward-pass overview

Figure 5.1 quantifies the landmark forward-pass gap between desktop and Redmi execution at BS=1. Batch size one means that the landmark network receives one detected face crop at a time, which is the most natural setting for an interactive camera application where frames arrive one by one. The chart should be read as a model-level diagnostic: it shows whether the landmark network itself is expensive, independent of detector, crop generation, and post-processing.

The gap is model-dependent. PFLD-MNv2-0.25 is already small on Redmi (9.99 ms in FP32 and 7.10 ms in INT8), while MobileFaceNet and PipNet-R18 remain much heavier. PipNet-R18 falls from 351.97 ms to 97.46 ms after INT8 quantization on Redmi, which is a large improvement but still far slower than the PC INT8 forward pass at 11.35 ms.

Figure 5.1 BS=1 median landmark-inference latency: PC vs Redmi. This measures only the landmark model forward pass in milliseconds, so lower bars are better.



5.3 Three-device BS=1 comparison

These BS=1 tables include all available backend/variant rows: PC PyTorch, PC XNNPACK FP32/INT8, iPhone Core ML, iPhone XNNPACK FP32/INT8, iPhone portable, and Redmi XNNPACK FP32/INT8. A dash means that the backend is not available or was not measured for that device and model.

Table 5.1 is the main BS=1 cross-device comparison because it isolates the landmark forward pass. PC XNNPACK and iPhone Core ML are the fastest paths for most models, while Redmi XNNPACK is slower but still shows clear INT8 gains for the heavier models. On Redmi, PFLD-MNv2-1.0 drops from 61.54 ms to 23.78 ms and PipNet-R18 from 351.97 ms to 97.46 ms after INT8 quantization. iPhone Core ML is much faster for the landmark network itself, reaching sub-millisecond latency for MobileFaceNet and PFLD variants.

5.4 Mobile-only BS=1 comparison

Figure 5.2 and Table 5.2 show that iPhone 14 has lower XNNPACK landmark latency than Redmi for every shared model and precision in this benchmark set. The size of the gap depends on the model: PFLD-MNv2-0.25 is already small on both phones, while PipNet-R18 is heavy on Redmi even after INT8 quantization. This section is intentionally about the common XNNPACK CPU path; the later iPhone section discusses Core ML separately.

Table 5.1 BS=1 landmark-inference latency across all available device/backend variants. This table isolates only the landmark model forward pass, so it explains whether the model itself is the bottleneck; lower is better. Bold marks the best value in each row.

Model	PC PyTorch	PC XNN FP32	PC XNN INT8	iPhone Core ML	iPhone XNN FP32	iPhone XNN INT8	iPhone Portable
MobileFaceNet	45.40	13.14	12.76	**0.60**	11.30	9.00	464.74
PFLD-MNv2-0.25	–	1.53	1.30	0.34	0.49	**0.26**	44.71
PFLD-MNv2-1.0	17.69	3.96	2.23	**0.49**	4.98	1.47	577.76
PipNet-R18	12.55	47.46	11.35	**1.43**	44.45	9.66	3847.09

Model	Redmi XNN FP32	Redmi XNN INT8
MobileFaceNet	184.33	178.39
PFLD-MNv2-0.25	9.99	7.10
PFLD-MNv2-1.0	61.54	23.78
PipNet-R18	351.97	97.46

Figure 5.2 BS=1 XNNPACK landmark-inference latency on iPhone 14 and Redmi. The chart uses only mobile devices and only the landmark forward pass; lower latency is better.

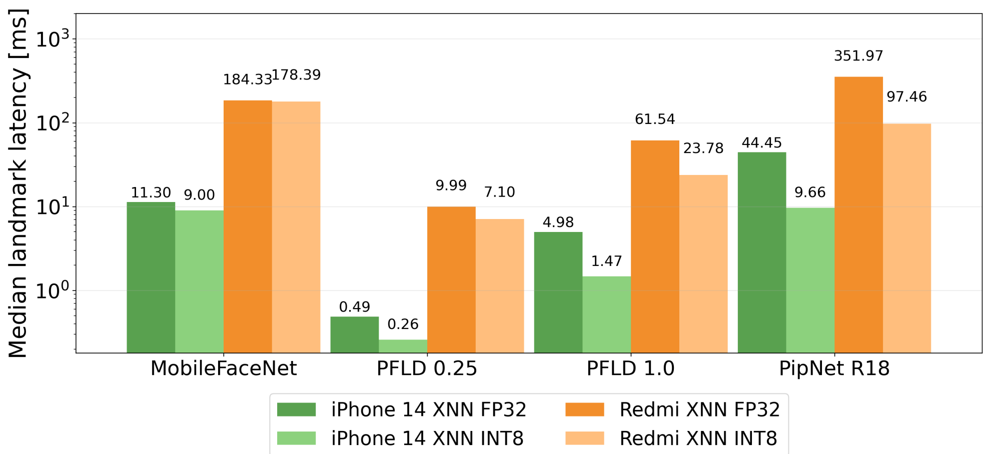


Table 5.2 BS=1 mobile-only XNNPACK landmark-inference latency; lower latency is better. Bold marks the lowest latency in each row.

Model	iPhone FP32 [ms]	iPhone INT8 [ms]	Redmi FP32 [ms]	Redmi INT8 [ms]
MobileFaceNet	11.30	**9.00**	184.33	178.39
PFLD-MNv2-0.25	0.49	**0.26**	9.99	7.10
PFLD-MNv2-1.0	4.98	**1.47**	61.54	23.78
PipNet-R18	44.45	**9.66**	351.97	97.46

Figure 5.3 Redmi: BS=1 landmark-inference latency by backend and quantization. This chart focuses on the landmark model forward pass; lower latency is better.

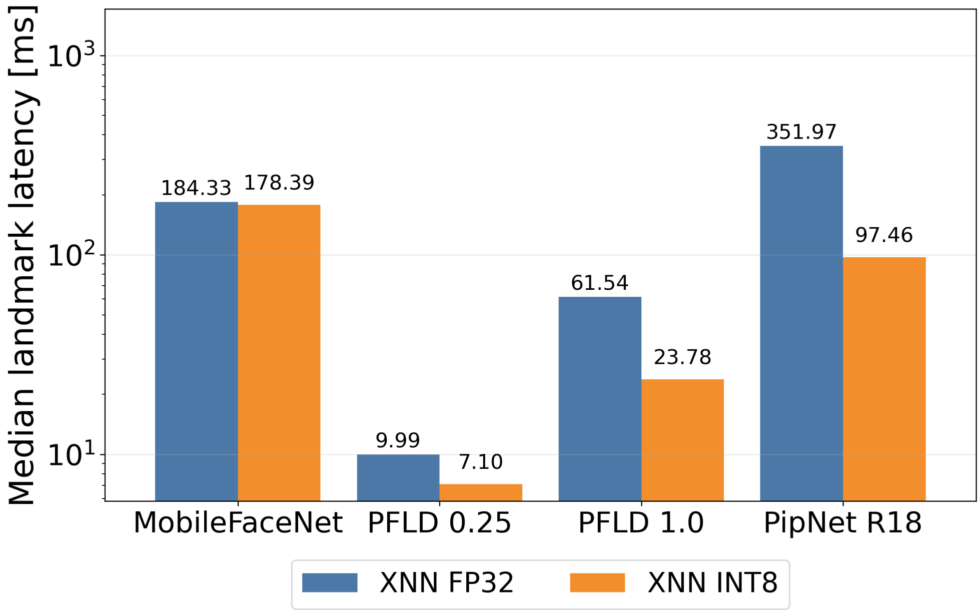
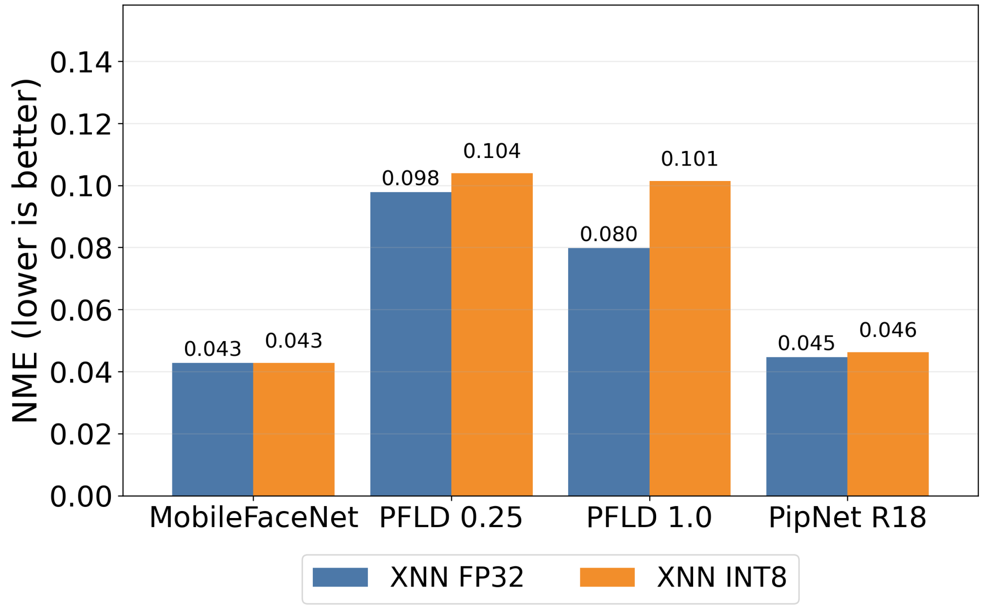


Table 5.3 Redmi BS=1 XNNPACK landmark latency and accuracy. LM inf. means landmark inference and ms means milliseconds. Bold values mark the lower latency and lower error within each model pair.

Model	Quant.	LM inf. [ms]	NME
MobileFaceNet	FP32	184.33	**0.0428**
MobileFaceNet	INT8	**178.39**	**0.0428**
PFLD-MNv2-0.25	FP32	9.99	**0.0978**
PFLD-MNv2-0.25	INT8	**7.10**	0.1039
PFLD-MNv2-1.0	FP32	61.54	**0.0798**
PFLD-MNv2-1.0	INT8	**23.78**	0.1014
PipNet-R18	FP32	351.97	**0.0446**
PipNet-R18	INT8	**97.46**	0.0463

Figure 5.4 Redmi: BS=1 NME accuracy by backend and quantization (lower is better).



5.5 Redmi BS=1 backend and quantization

Figure 5.3 and Table 5.3 show that the usefulness of INT8 on Redmi depends on how much landmark computation the model contains. PipNet-R18 landmark inference drops from 351.97 ms to 97.46 ms, a 3.6× reduction. PFLD-MNv2-1.0 drops from 61.54 ms to 23.78 ms, a 2.6× reduction. The smaller models improve only slightly, so their bottleneck is less dominated by quantizable landmark computation.

The accuracy picture is correspondingly model-dependent. NME for MobileFaceNet is essentially unchanged (0.0428 to 0.0428, $< 0.1\%$ relative), and PipNet-R18 only degrades by ~3.7% relative (0.0446 to 0.0463). The PFLD family is more sensitive. PFLD-MNv2-0.25 loses ~6% relative NME (0.0978 to 0.1039), and the heavier PFLD-MNv2-1.0 loses ~27% relative (0.0798 to 0.1014). The PFLD-MNv2-1.0 case is the canonical latency/accuracy trade-off in this benchmark: a 2.6× landmark-latency reduction costs a 27% NME increase, which may or may not be acceptable depending on the actual use case. PipNet-R18 INT8, by contrast, is a near-Pareto win. It has the largest landmark latency gain in the matrix with only a barely measurable accuracy loss, making it the strongest INT8 candidate among the tested models on this device.

5.6 Redmi batch scaling

Figure 5.5 Redmi landmark-inference latency batch scaling for PFLD-MNv2-1.0 using XNNPACK variants. Lower latency is better.

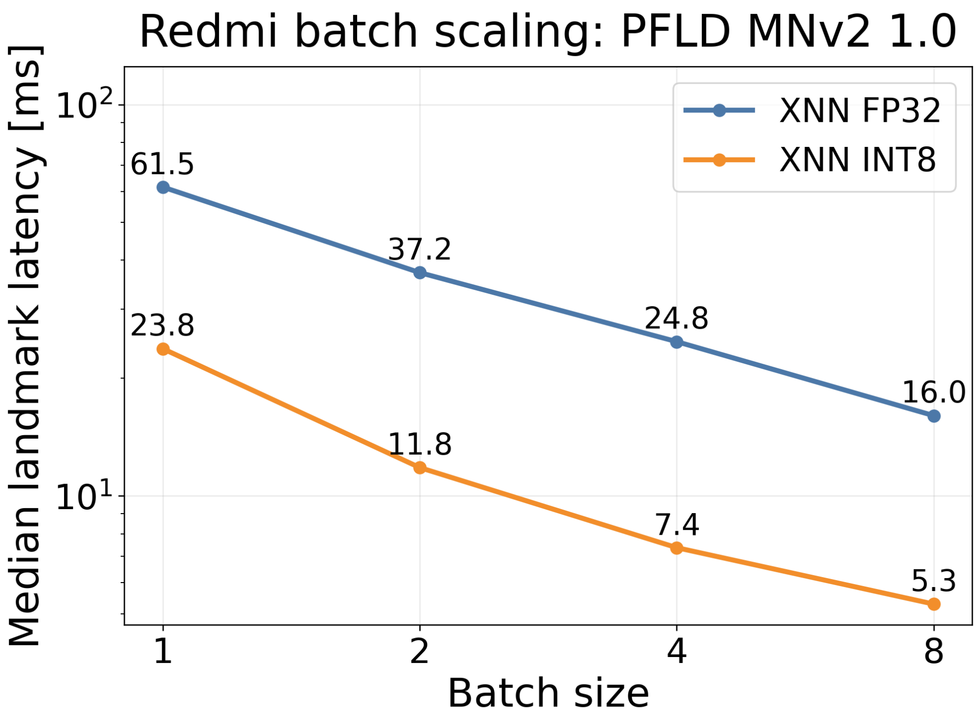
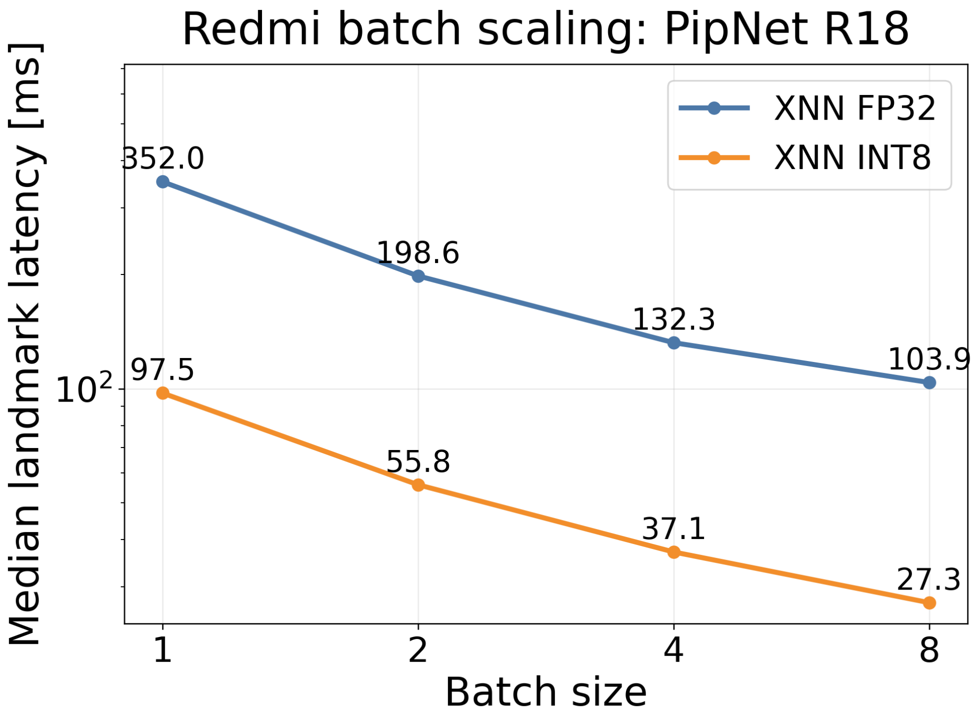


Table 5.4 Redmi batch scaling in landmark-inference latency. LM means landmark inference and BS means batch size. Bold marks the lowest latency in each batch-size column.

Model	Quant.	LM [ms] BS=1	LM [ms] BS=2	LM [ms] BS=4	LM [ms] BS=8
PFLD-MNv2-1.0	FP32	61.54	37.19	24.78	16.03
PFLD-MNv2-1.0	INT8	**23.78**	**11.81**	**7.37**	**5.29**
PipNet-R18	FP32	351.97	198.58	132.34	103.89
PipNet-R18	INT8	97.46	55.81	37.10	27.26

Figures 5.5 and 5.6 show the shape of the landmark-latency batch-scaling curves for Redmi, while Table 5.4 provides the exact values. The landmark forward pass improves strongly as the resolved landmark batch grows: PFLD-MNv2-1.0 INT8 drops from 23.78 ms to 5.29 ms (4.5× across 8× batch), and PipNet-R18 INT8 drops from 97.46 ms to 27.26 ms (3.6×). This is the batch-scaling result that matters for the landmark models themselves; run FPS can

Figure 5.6 Redmi landmark-inference latency batch scaling for PipNet ResNet18 using XNNPACK variants. Lower latency is better.



remain flat if the detector or image handling is the dominant part of the full pipeline.

Table 5.5 Three-device INT8 XNNPACK batch scaling for PFLD-MNv2-1.0 in landmark-inference latency; lower is better. Bold marks the best value in each batch-size column.

Device	LM [ms] BS=1	LM [ms] BS=2	LM [ms] BS=4	LM [ms] BS=8
PC	2.23	0.77	0.55	0.43
iPhone 14	**1.47**	**0.72**	**0.36**	**0.17**
Redmi	23.78	11.81	7.37	5.29

Table 5.6 Three-device INT8 XNNPACK batch scaling for PipNet-R18 in landmark-inference latency; lower is better. Bold marks the best value in each batch-size column.

Device	LM [ms] BS=1	LM [ms] BS=2	LM [ms] BS=4	LM [ms] BS=8
PC	11.35	10.24	7.20	5.57
iPhone 14	**9.66**	**4.82**	**2.46**	**1.22**
Redmi	97.46	55.81	37.10	27.26

Tables 5.5 and 5.6 compare batch scaling across all three devices. A larger batch gives the landmark model more face crops at once, which can improve hardware utilization. This is useful when several faces are present in one frame or when an offline process can group multiple images together. However, it is less useful for a single-face real-time camera stream, because the detector still runs on incoming frames and the application cannot wait too long just to fill a batch.

For both models, iPhone 14 has the lowest landmark latency in the INT8 XNNPACK batch-scaling comparison, while Redmi is slower but improves consistently with larger batches.

5.7 Quantization deltas and cumulative error behavior

Figure 5.7 must be read in the opposite direction from the latency tables. Here, values near zero are good because they mean INT8 changed the error very little. Positive values mean the INT8 model is less accurate than its FP32 version. The cumulative histograms in Figures 5.8 and 5.9 provide the deeper view behind those averages. If the FP32 and INT8 curves overlap, quantization does not substantially change the distribution of easy and difficult images.

Figure 5.7 Redmi: INT8 vs FP32 NME difference at BS=1.

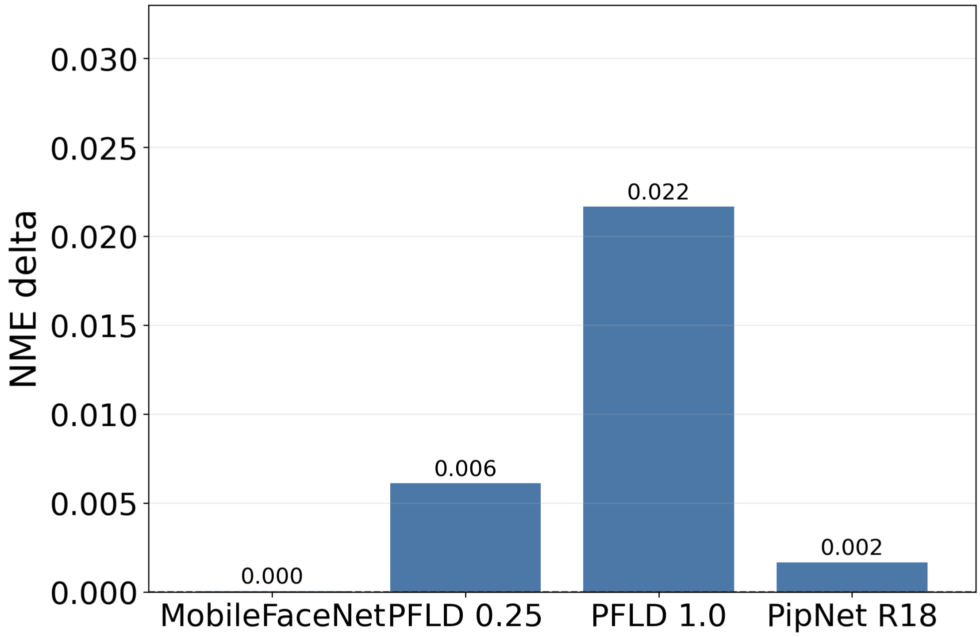
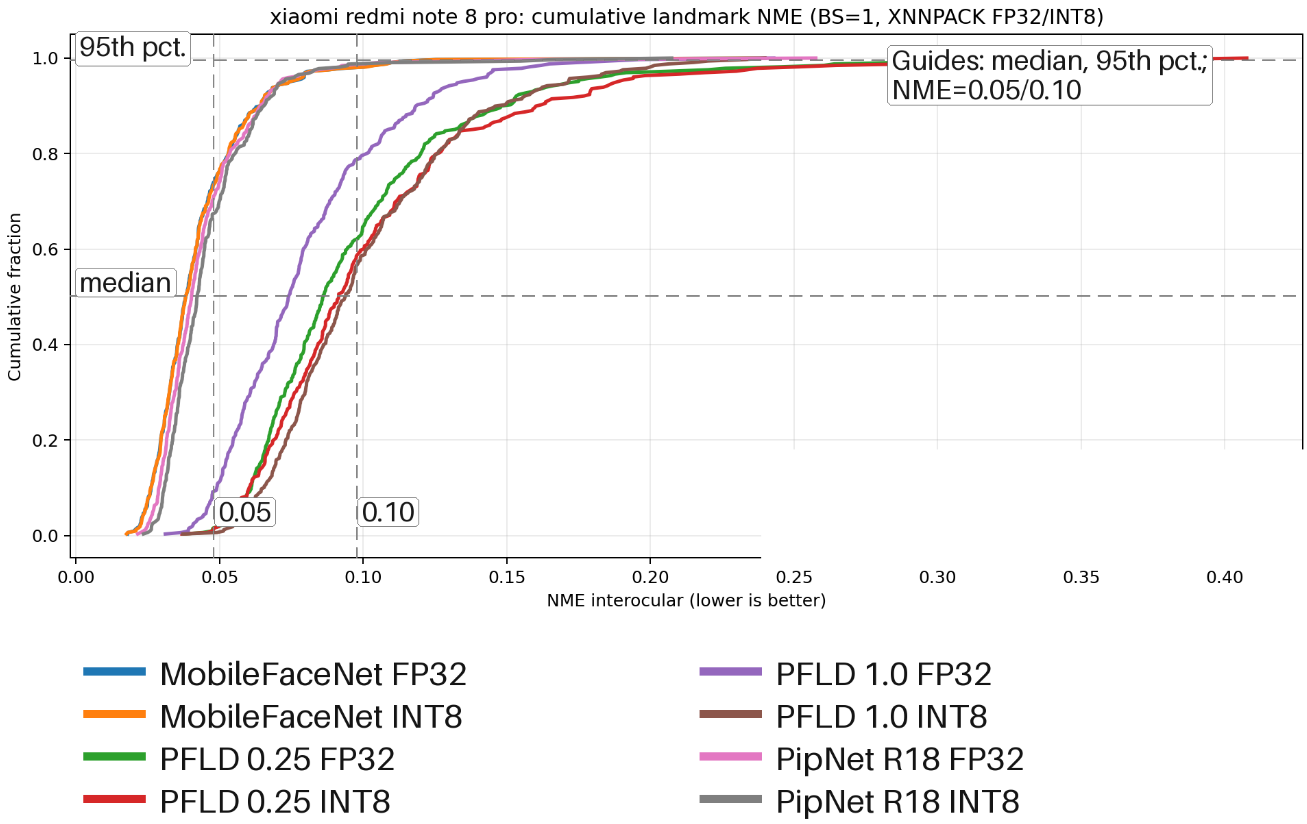


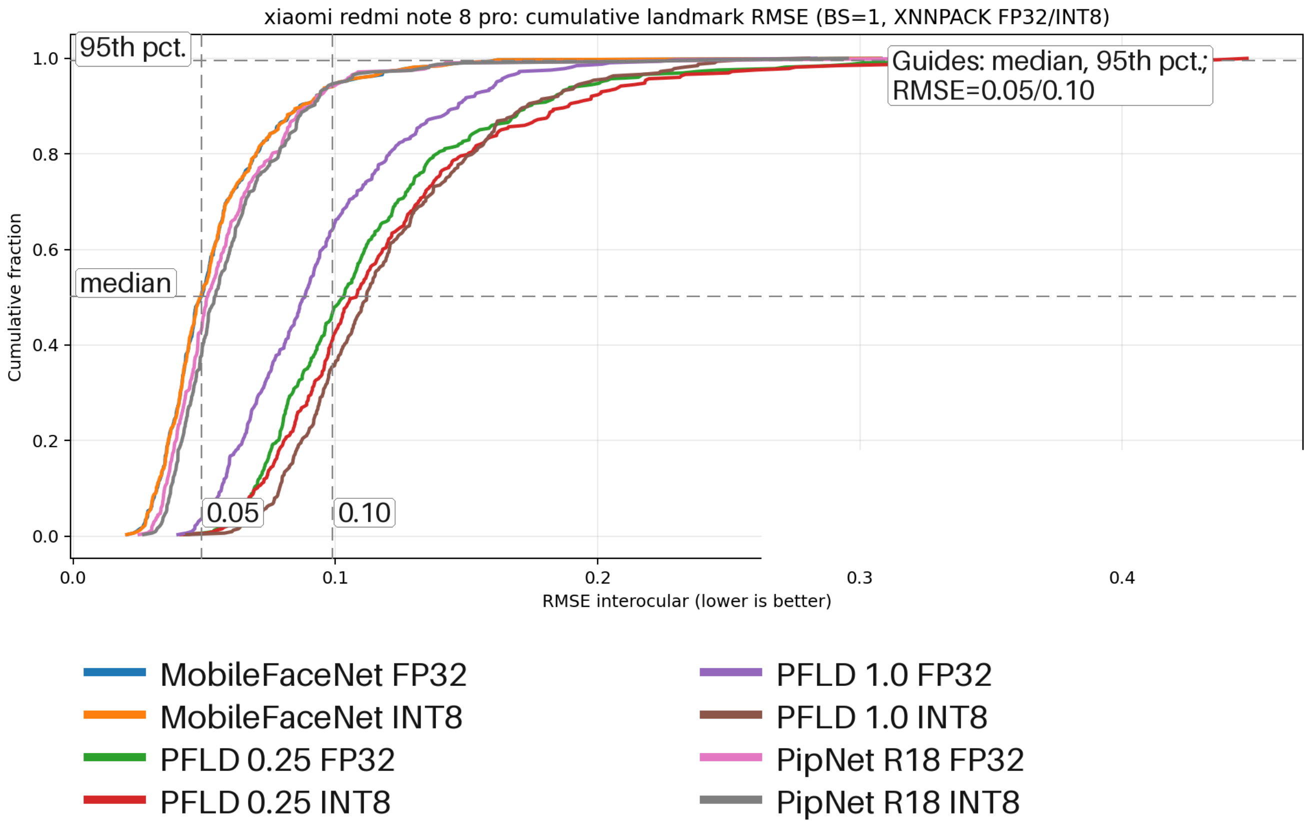
Table 5.7 BS=1 PC INT8 and Redmi INT8 landmark-inference latency. Bold marks the lower latency in each row.

Model	PC INT8 LM [ms]	Redmi INT8 LM [ms]
MobileFaceNet	**12.76**	178.39
PFLD-MNv2-0.25	**1.30**	7.10
PFLD-MNv2-1.0	**2.23**	23.78
PipNet-R18	**11.35**	97.46

Figure 5.8 Redmi cumulative NME histogram for BS=1 XNNPACK FP32 and INT8 runs. Curves that rise earlier and stay higher indicate that more images have small normalized landmark error.



■ **Figure 5.9** Redmi cumulative RMSE histogram for BS=1 XNNPACK FP32 and INT8 runs. This uses the same cumulative logic as the NME chart, but reports root-mean-square landmark error.



If the INT8 curve shifts to the right or rises more slowly, more images require a larger error threshold before they count as successful. In these charts, MobileFaceNet and PipNet-R18 preserve similar distributions after quantization, whereas PFLD-MNv2-1.0 separates more visibly. The deployment heuristic is therefore: prefer INT8 for PipNet-R18 when latency is important, treat MobileFaceNet INT8 as a small but mostly free latency win, and evaluate the PFLD-family accuracy/latency trade-off against the needs of the actual application.

Table 5.7 shows the absolute landmark-latency deployment gap. Even after INT8 quantization on both sides, the PC landmark forward pass is substantially faster than Redmi, especially for MobileFaceNet and PipNet-R18.

5.8 iPhone 14 result interpretation

Figure 5.10 iPhone 14: BS=1 median landmark-inference latency.

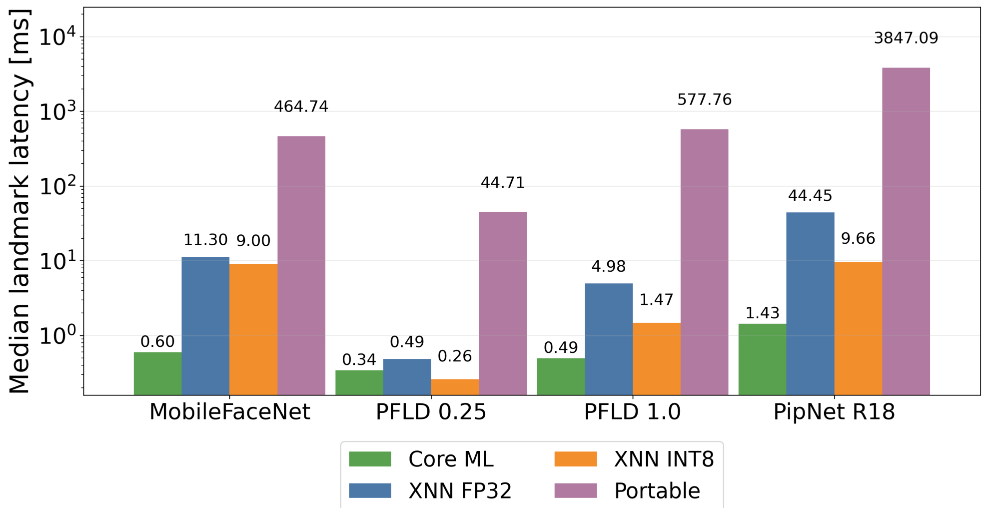


Table 5.8 iPhone 14 BS=1 landmark-inference latency by backend. XNN means XNNPACK. Bold marks the lowest latency in each row.

Model	Core ML LM [ms]	XNN FP32 LM [ms]	XNN INT8 LM [ms]	Portable LM [ms]
MobileFaceNet	**0.60**	11.30	9.00	464.74
PFLD-MNv2-0.25	0.34	0.49	**0.26**	44.71
PFLD-MNv2-1.0	**0.49**	4.98	1.47	577.76
PipNet-R18	**1.43**	44.45	9.66	3847.09

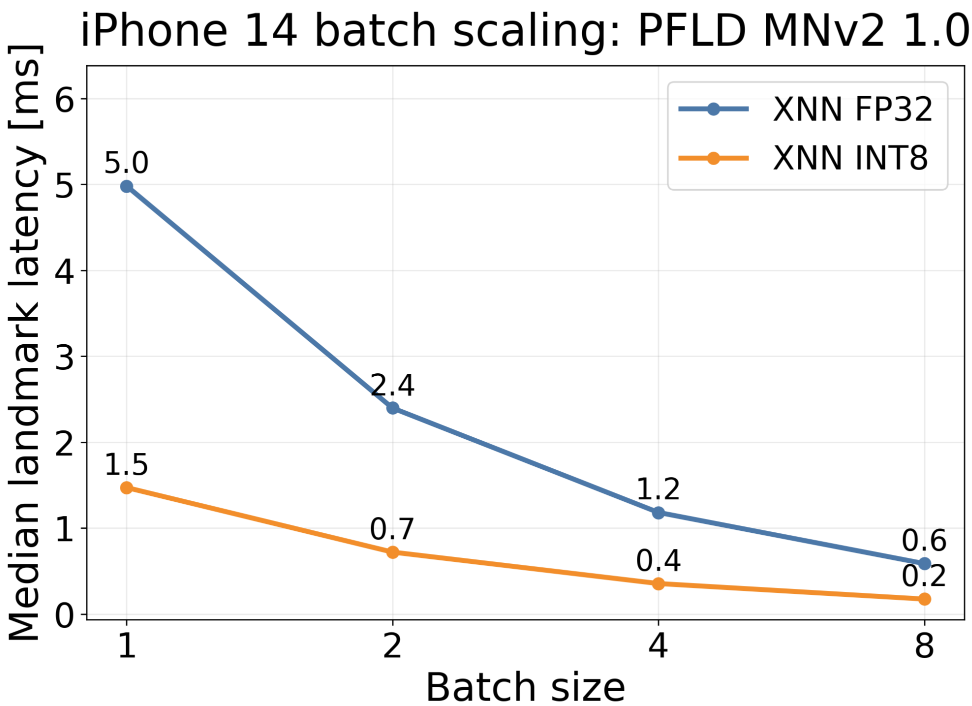
Figure 5.10 and Table 5.8 show the difference between iPhone backend delegates for the landmark forward pass. Core ML reduces most landmark models

to sub-millisecond or near-sub-millisecond latency, with PipNet-R18 still only 1.43 ms. This is a different operating point from XNNPACK, where landmark inference remains measurable and therefore still benefits from quantization.

Comparing XNNPACK with Portable shows the cost of giving up backend-specific acceleration. Portable is 17–97× slower than Core ML on the same model, ranging from 17.3× for PFLD-MNv2-0.25 to 97.1× for PipNet-R18. Portable is therefore unsuitable for any deployed configuration; it is only useful as a correctness baseline.

The INT8/FP32 split on XNNPACK follows the same model-dependent pattern as on Redmi: PipNet-R18 is again a large winner (44.45 → 9.66 ms, 4.6×), PFLD-MNv2-1.0 improves from 4.98 ms to 1.47 ms (3.4×), and the smaller models move less.

Figure 5.11 iPhone 14 landmark-inference latency batch scaling for PFLD-MNv2-1.0.



The iPhone 14 batch-scaling pattern is qualitatively the same as on Redmi. PFLD-MNv2-1.0 INT8 lands at 0.17 ms per image at BS=8, and PipNet-R18 INT8 at 1.22 ms; in both cases the landmark forward has effectively been amortised away.

Table 5.10 sets the deployment context. With the same INT8 XNNPACK configuration on both sides, iPhone 14 has lower landmark-forward latency than the PC for all four selected models in this dataset.

Figure 5.12 iPhone 14 landmark-inference latency batch scaling for PipNet ResNet18.

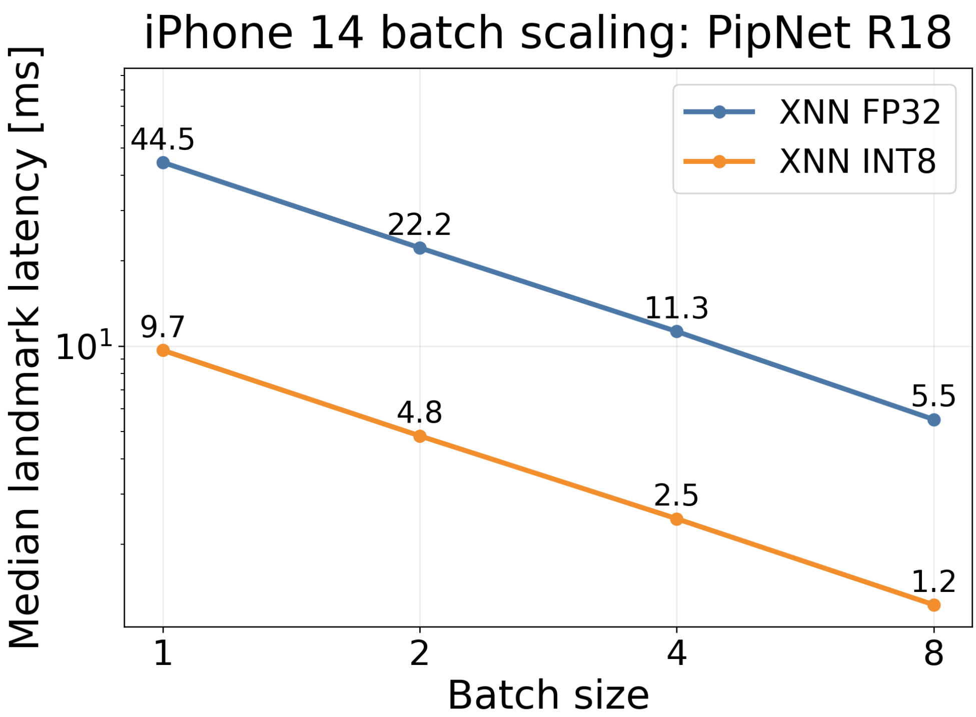


Table 5.9 iPhone 14 batch scaling in landmark-inference latency. LM means landmark inference and BS means batch size. Bold marks the lowest latency in each batch-size column.

Model	Quant.	LM [ms] BS=1	LM [ms] BS=2	LM [ms] BS=4	LM [ms] BS=8
PFLD-MNv2-1.0	FP32	4.98	2.40	1.18	0.59
PFLD-MNv2-1.0	INT8	**1.47**	**0.72**	**0.36**	**0.17**
PipNet-R18	FP32	44.45	22.20	11.26	5.51
PipNet-R18	INT8	9.66	4.82	2.46	1.22

Table 5.10 BS=1 PC INT8 and iPhone 14 INT8 landmark-inference latency. Bold marks the lower latency in each row.

Model	PC INT8 LM [ms]	iPhone INT8 LM [ms]
MobileFaceNet	12.76	**9.00**
PFLD-MNv2-0.25	1.30	**0.26**
PFLD-MNv2-1.0	2.23	**1.47**
PipNet-R18	11.35	**9.66**

Chapter 6
Conclusion

This thesis presented a full benchmark for landmark detection models on mobile devices with ExecuTorch, from theoretical background and export details to cross platform runtime evaluation.

The thesis delivered:

- implementation and comparison of mobile inference paths on both iOS and Android,
- evaluation of landmark forward-pass latency and full-pipeline metrics,
- multi-device comparison with reproducible logging and offline analysis,
- quantization study (INT8 PTQ) with speed/accuracy trade-off analysis,
- optional energy-related telemetry as auxiliary data where available.

The results show four main points:

- backend and quantization selection strongly influence runtime;
- INT8 can provide major landmark-latency gains for heavier landmark models, but accuracy impact depends on the model;
- increasing batch size improves per-image landmark-inference latency
- PC reference runs are useful for capacity comparison

Overall, the implemented methodology and artifact pipeline provide a reproducible base for future iterations, including model replacement, backend retuning, and extension to more mobile devices.

Appendix A

Reproducibility

This appendix lists the commands used to regenerate the CSV data, charts, and tables referenced in the thesis.

A.1 Combined CSV and chart generation

From the repository root:

```
./.venv/bin/python benchmarks/analysis/plot_runs_dashboard.py \
  --mobile-root benchmarks/results/mobile \
  --pc-root benchmarks/results/pc/runs \
  --focus-device "xiaomi redmi note 8 pro" \
  --max-resolved-batch 8 \
  --output-csv \
    benchmarks/results/combined/combined_runs_from_json.csv \
  --output-dir benchmarks/results/combined/charts
```

A.2 Summary table generation

```
./.venv/bin/python benchmarks/analysis/summarize_benchmarks.py \
  --input benchmarks/results/combined/combined_runs_from_json.csv \
  --output-dir benchmarks/results/combined/tables \
  --focus-device "xiaomi redmi note 8 pro"
```

A.3 Readable thesis chart regeneration

```
./.venv/bin/python \
  benchmarks/analysis/generate_readable_thesis_charts.py
```

A.4 INT8 XNNPACK landmark model export with post-training quantization

```
EXPORT_BATCH_SIZES=1 \
EXPORT_DYNAMIC_BATCH=1 \
EXPORT_DYNAMIC_MIN_BATCH=1 \
EXPORT_DYNAMIC_MAX_BATCH=128 \
XNNPACK_INT8_CALIBRATION_SAMPLES=32 \
XNNPACK_INT8_METHODS=pc,pt \
./models/export_landmark_models_xnnpack_quantized.sh
```

Optional MobileFaceNet stability switch (enabled by default):

```
MOBILEFACENET_STATIC_INT8_DYNAMIC_ACT_WORKAROUND=1
```

Bibliography

1. WANG, Siqi; PATHANIA, Anuj; MITRA, Tulika. Neural Network Inference on Mobile SoCs. *IEEE Design & Test*. 2020, vol. 37, no. 5, pp. 50–57. Available from DOI: `10.1109/MDAT.2020.2968258`.
2. LI, Zhuojin; PAOLIERI, Marco; GOLUBCHIK, Leana. A Benchmark for ML Inference Latency on Mobile Devices. In: *Proceedings of the 7th International Workshop on Edge Systems, Analytics and Networking (EdgeSys '24)*. 2024. Available from DOI: `10.1145/3642968.3654818`.
3. CHEN, Sheng; LIU, Yang; GAO, Xiang; HAN, Zhen. MobileFaceNets: Efficient CNNs for Accurate Real-Time Face Verification on Mobile Devices. In: *Biometric Recognition (CCBR 2018)*. Springer, 2018, vol. 10996, pp. 428–438. Lecture Notes in Computer Science. Available from DOI: `10.1007/978-3-319-97909-0_46`.
4. ZHU, Xiangxin; RAMANAN, Deva. Face Detection, Pose Estimation, and Landmark Localization in the Wild. In: *2012 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2012, pp. 2879–2886. Available from DOI: `10.1109/CVPR.2012.6248014`.
5. GUO, Xiaojie; LI, Siyuan; YU, Jinke; ZHANG, Jiawan; MA, Jiayi; MA, Lin; LIU, Wei; LING, Haibin. PFLD: A Practical Facial Landmark Detector. *arXiv preprint arXiv:1902.10859*. 2019. Available from DOI: `10.48550/arXiv.1902.10859`.
6. VROCHIDOU, Eleni; PAPIĆ, Vladan; KALAMPOKAS, Theofanis; PAPAKOSTAS, George A. Automatic Facial Palsy Detection—From Mathematical Modeling to Deep Learning. *Axioms*. 2023, vol. 12, no. 12, p. 1091. Available from DOI: `10.3390/axioms12121091`.
7. REDDI, Vijay Janapa; KANTER, David; MATTSON, Peter; DUKE, Jared; NGUYEN, Thai; CHUKKA, Ramesh; SHIRING, Ken; TAN, Koan-Sin; CHARLEBOIS, Mark; CHOU, William, et al. MLPerf Mobile Inference Benchmark: An Industry-Standard Open-Source Machine Learning Benchmark for On-Device AI. *Proceedings of Machine Learning and Sys-*

tems [online]. 2022, vol. 4, pp. 352–369 [visited on 2026-04-19]. Available from: https://proceedings.mlsys.org/paper_files/paper/2022/hash/a2b2702ea7e682c5ea2c20e8f71efb0c-Abstract.html.

8. LEE, Juhyun; CHIRKOV, Nikolay; IGNASHEVA, Ekaterina; PISARCHYK, Yury; SHIEH, Mogan; RICCARDI, Fabio; SAROKIN, Raman; KULIK, Andrei; GRUNDMANN, Matthias. On-Device Neural Net Inference with Mobile GPUs. *arXiv preprint arXiv:1907.01989*. 2019. Available from DOI: 10.48550/arXiv.1907.01989.
9. PYTORCH FOUNDATION. *XNNPACK Backend — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/backends/xnnpack/xnnpack-overview.html.
10. GOOGLE. *XNNPACK: High-efficiency Neural Network Operators* [online]. 2026. [visited on 2026-04-14]. Available from: https://github.com/google/XNNPACK.
11. APPLE. *Core ML* [online]. 2026. [visited on 2026-04-14]. Available from: https://developer.apple.com/documentation/coreml.
12. PYTORCH FOUNDATION. *Core ML Backend — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/backends/coreml/coreml-overview.html.
13. PYTORCH FOUNDATION. *How ExecuTorch Works* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/intro-how-it-works.html.
14. PYTORCH FOUNDATION. *Architecture and Components — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/getting-started-architecture.html.
15. HOWARD, Andrew G.; ZHU, Menglong; CHEN, Bo; KALENICHENKO, Dmitry; WANG, Weijun; WEYAND, Tobias; ANDREETTO, Marco; ADAM, Hartwig. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv preprint arXiv:1704.04861*. 2017. Available from DOI: 10.48550/arXiv.1704.04861.
16. SANDLER, Mark; HOWARD, Andrew; ZHU, Menglong; ZHMOGINOV, Andrey; CHEN, Liang-Chieh. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 4510–4520. Available from DOI: 10.1109/CVPR.2018.00474.

17. ZHANG, Xiangyu; ZHOU, Xinyu; LIN, Mengxiao; SUN, Jian. ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 6848–6856. Available from DOI: 10.1109/CVPR.2018.00716.

18. IANDOLA, Forrest N.; MOSKEWICZ, Matthew W.; ASHRAF, Khalid; HAN, Song; DALLY, William J.; KEUTZER, Kurt. SqueezeNet: AlexNet-Level Accuracy with 50x Fewer Parameters and <1MB Model Size. *arXiv preprint arXiv:1602.07360*. 2016. Available from DOI: 10.48550/arXiv.1602.07360.

19. HAN, Song; MAO, Huizi; DALLY, William J. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. *arXiv preprint arXiv:1510.00149*. 2016. Available from DOI: 10.48550/arXiv.1510.00149.

20. JACOB, Benoit; KLIGYS, Skirmantas; CHEN, Bo; ZHU, Menglong; TANG, Matthew; HOWARD, Andrew; ADAM, Hartwig; KALENICHENKO, Dmitry. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 2704–2713. Available from DOI: 10.1109/CVPR.2018.00286.

21. KRISHNAMOORTHI, Raghuraman. Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper. *arXiv preprint arXiv:1806.08342*. 2018. Available from DOI: 10.48550/arXiv.1806.08342.

22. PYTORCH AO CONTRIBUTORS. *PT2E Quantization Tutorials and Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://docs.pytorch.org/ao/main/tutorials_source/pt2e_quant_ptq.html.

23. DENG, Jiankang; GUO, Jia; VERVERAS, Evangelos; KOTSIA, Irene; ZAFEIRIOU, Stefanos. RetinaFace: Single-Stage Dense Face Localisation in the Wild. *arXiv preprint arXiv:1905.00641*. 2020. Available from DOI: 10.48550/arXiv.1905.00641.

24. BIUBUG6. *Pytorch RetinaFace* [online]. 2026. [visited on 2026-04-29]. Available from: https://github.com/biubug6/Pytorch_Retinaface.

25. ZHANG, Shifeng; ZHU, Xiangyu; LEI, Zhen; SHI, Hailin; WANG, Xiaobo; LI, Stan Z. FaceBoxes: A CPU Real-Time Face Detector with High Accuracy. In: *2017 IEEE International Joint Conference on Biometrics (IJCB)*. 2017, pp. 1–9. Available from DOI: 10.1109/BTAS.2017.8272675.

26. JHB86253817. *FaceBoxesV2* [online]. 2026. [visited on 2026-04-29]. Available from: https://github.com/jhb86253817/FaceBoxesV2.

27. GITHUB-LUFFY. *PFLD 68points Pytorch* [online]. 2026. [visited on 2026-04-29]. Available from: https://github.com/github-luffy/PFLD_68points_Pytorch.
28. XIAOCCER. *MobileFaceNet Pytorch* [online]. 2026. [visited on 2026-04-29]. Available from: https://github.com/Xiaoccer/MobileFaceNet_Pytorch.
29. JIN, Haibo; LIAO, Shengcai; SHAO, Ling. Pixel-in-Pixel Net: Towards Efficient Facial Landmark Detection in the Wild. *International Journal of Computer Vision*. 2021, vol. 129, no. 12, pp. 3174–3194. Available from DOI: 10.1007/s11263-021-01521-4.
30. JHB86253817. *PIPNet* [online]. 2026. [visited on 2026-04-29]. Available from: https://github.com/jhb86253817/PIPNet.
31. TENSORFLOW. *TensorFlow Lite* [online]. 2026. [visited on 2026-04-29]. Available from: https://www.tensorflow.org/lite.
32. MICROSOFT. *Deploy on Mobile — ONNX Runtime Documentation* [online]. 2026. [visited on 2026-04-29]. Available from: https://onnxruntime.ai/docs/tutorials/mobile/.
33. PYTORCH FOUNDATION. *torch.export User Guide* [online]. 2026. [visited on 2026-04-14]. Available from: https://docs.pytorch.org/docs/stable/user_guide/torch_compiler/export.html.
34. PYTORCH FOUNDATION. *Model Export and Lowering — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/using-executorch-export.html.
35. PYTORCH FOUNDATION. *.pte file format — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/pte-file-format.html.
36. PYTORCH FOUNDATION. *Understanding Backends and Delegates — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/compiler-delegate-and-partitioner.html.
37. PYTORCH FOUNDATION. *Memory Planning — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/compiler-memory-planning.html.
38. PYTORCH FOUNDATION. *Using ExecuTorch on Android* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/using-executorch-android.html.
39. PYTORCH FOUNDATION. *Using ExecuTorch on iOS* [online]. 2026. [visited on 2026-04-14]. Available from: https://pytorch.org/executorch/stable/using-executorch-ios.html.

40. PYTORCH FOUNDATION. *.ptd file format — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: `https://pytorch.org/executorch/stable/ptd-file-format.html`.

41. PYTORCH FOUNDATION. *XNNPACK Quantization — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: `https://pytorch.org/executorch/stable/backends/xnnpack/xnnpack-quantization.html`.

42. PYTORCH FOUNDATION. *Quantization Overview — ExecuTorch Documentation* [online]. 2026. [visited on 2026-04-14]. Available from: `https://pytorch.org/executorch/stable/quantization-overview.html`.

Contents of the attachment

The electronic attachment contains the source code of the benchmark applications, benchmarking scripts, collected run exports and the thesis sources.

```
/
├── android/ ........................ Android benchmark app source code
├── ios/ ................................ iOS benchmark app source code
├── benchmarks/ ......... Benchmark harness, analysis scripts, and results
│   ├── analysis/ .... JSON aggregation, charting and table export scripts
│   └── results/ ....... Collected exported runs + generated charts/tables
├── models/ .......... Detector and landmark model files used by the apps
└── thesis/ ................................. LaTeX sources of the thesis
    ├── ctufit-thesis.tex ...................... Main thesis entry point
    └── text/ ........................................... Chapter sources
```